

使用 Cloud Run functions 的修訂版本，執行流量拆分、漸進式推出和復原作業

來源：<https://codelabs.developers.google.com/codelabs/revisions-2nd-gen-cloud-functions-traffic-splitting-gradual-rollout-rollbacks?hl=zh-tw>

步驟數：7

瞭解如何在 Cloud Run 函式中使用修訂版本，進行流量拆分、漸進式推出和復原作業。

目錄

1. [1. 簡介](#)
2. [2. 設定和需求](#)
3. [3. 流量分配](#)
4. [4. 逐步推出](#)
5. [5. 復原](#)
6. [6. 恭喜！](#)
7. [7. 清除所用資源](#)

步驟 1 · 約 0 分鐘

1. 簡介

總覽

[Cloud Run functions](#) 可讓您指定應接收流量的修訂版本，以及修訂版本接收的流量百分比。您可以透過修訂版本復原至先前的版本、漸進式推出修訂版本，並在多個修訂版本之間拆分流量。

本程式碼實驗室會說明如何使用修訂版本管理 Cloud Run 函式的流量。如要進一步瞭解修訂版本，請參閱 [Cloud Run 說明文件](#)。

課程內容

- 如何在 Cloud Run 函式的兩個或多個修訂版本之間拆分流量
 - 如何逐步推出新修訂版本
 - 如何復原為先前的修訂版本
-

步驟 2 · 約 0 分鐘

2. 設定和需求

必要條件

- 您已登入 Cloud Console。
- 您先前已部署 Cloud Run 函式。舉例來說，您可以按照[部署 Cloud Run 函式](#)的步驟開始操作。

啟用 Cloud Shell

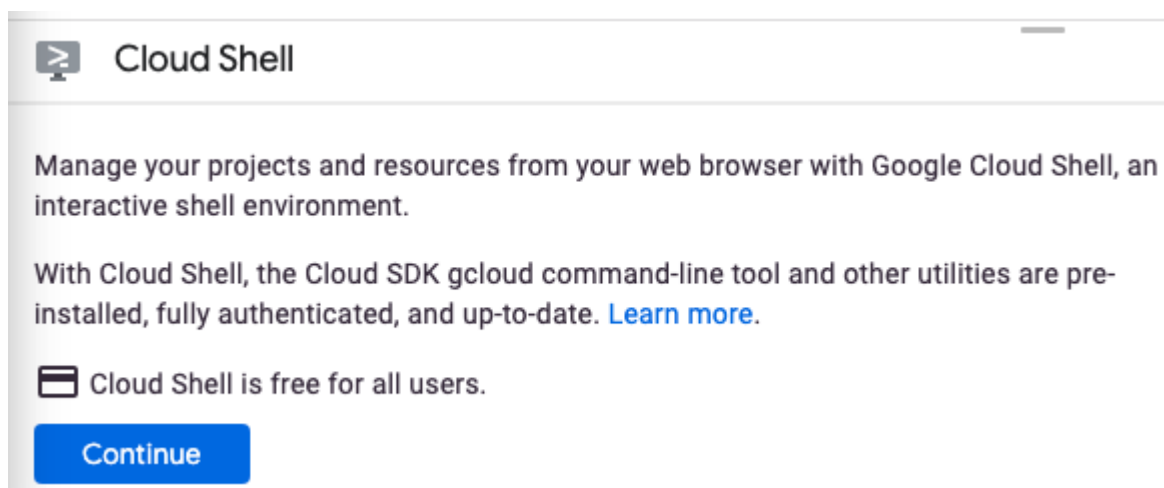
1. 在 Cloud 控制台，點選「啟用 Cloud Shell」圖示



。



如果您是首次啟動 Cloud Shell，系統會顯示中繼畫面，說明這個指令列環境。如果出現中繼畫面，請按一下「繼續」。



佈建並連至 Cloud Shell 預計只需要幾分鐘。



```
Welcome to Cloud Shell! For help, visit https://cloud.google.com/cloud-shell/help.  
testuser003@codelab-test003:~$ gcloud auth list
```

這部虛擬機器已載入所有必要的開發工具，並提供永久的 5 GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。本程式碼研究室幾乎所有工作都可在瀏覽器上完成。

連至 Cloud Shell 後，您應該會看到驗證已完成，專案也已設為獲派的專案 ID。

2. 在 Cloud Shell 中執行下列指令，確認您已通過驗證：

〇〇

```
gcloud auth list
```

指令輸出



```
Credentialed Accounts
ACTIVE  ACCOUNT
*       <my_account>@<my_domain.com>

To set the active account, run:
$ gcloud config set account `ACCOUNT`
```

注意： `gcloud` 指令列工具是 Google Cloud 中功能強大的整合式指令列工具，並已預先安裝於 Cloud Shell。你會發現它支援 Tab 鍵自動完成功能。第一次執行指令時，系統可能會提示您進行驗證。詳情請參閱 [gcloud 指令列工具總覽](#)。

3. 在 Cloud Shell 中執行下列指令，確認 `gcloud` 指令知道您的專案：

```
gcloud config list project
```

指令輸出

```
[core]
project = <PROJECT_ID>
```

如未設定，請輸入下列指令手動設定專案：

```
gcloud config set project <PROJECT_ID>
```

指令輸出

```
Updated property [core/project].
```

注意： 如果您在 Cloud Shell 以外的環境完成本教學課程，請按照「[設定應用程式預設憑證](#)」一文的說明操作。

步驟 3 · 約 0 分鐘

3. 流量分配

這個範例說明如何建立函式，讀取顏色環境變數，並使用該背景顏色回應修訂版本名稱。

雖然本程式碼研究室使用 Node.js，但您可以使用任何執行階段。

設定環境變數

您可以設定本程式碼實驗室全程都會使用的環境變數。

```
REGION=<YOUR_REGION>
PROJECT_ID=<YOUR-PROJECT-ID>
BG_COLOR=darkseagreen
```

建立函式

首先，請建立原始碼的目錄，然後 `cd` 到該目錄。

```
mkdir revisions-gcf-codelab && cd $_
```

接著，請建立 `package.json` 檔案，並加入以下內容：

```
{
  "dependencies": {
    "@google-cloud/functions-framework": "^3.0.0"
  }
}
```

接著，請建立 `index.js` 來源檔案，並加入下列內容：

```
const functions = require('@google-cloud/functions-framework');

const BG_COLOR = process.env.BG_COLOR;
const K_REVISION = process.env.K_REVISION;

functions.http('helloWorld', (req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/html' });
  res.end('<html><body style="background-color:' + BG_COLOR + ';"' + '><div><p>' +
    'Hello from ' + K_REVISION + ' using color ' + BG_COLOR + '</p></div></body>' +
    '</html>');
});
```

如要將 Cloud Run 函式直接部署至 Cloud Run，請執行下列指令：

```
gcloud beta run deploy traffic-splitting-gcf \  
  --source . \  
  --function helloWorld \  
  --region $REGION \  
  --no-allow-unauthenticated \  
  --set-env-vars BG_COLOR=$BG_COLOR
```

如要部署為 Cloud Functions 第 2 代，請使用下列指令：

```
gcloud functions deploy traffic-splitting-gcf \  
  --gen2 \  
  --runtime=nodejs20 \  
  --region=$REGION \  
  --source=. \  
  --entry-point=helloWorld \  
  --trigger-http \  
  --no-allow-unauthenticated \  
  --set-env-vars BG_COLOR=$BG_COLOR
```

如要測試函式，您可以捲動現有端點，查看 HTML 中的 darkseagreen 顏色，也可以使用瀏覽器直接點擊端點，查看背景顏色。

```
SERVICE_URL=$(gcloud run services describe traffic-splitting-gcf --platform managed --region $REGION --format 'value(status.url)')  
  
curl -H "Authorization: bearer $(gcloud auth print-identity-token)" -X GET $SERVICE_URL
```

現在部署第二個修訂版本，並將背景顏色設為棕褐色。

如要將 Cloud Run 函式直接部署至 Cloud Run，請執行下列指令：

```
# update the env var  
BG_COLOR=tan  
  
gcloud beta run deploy traffic-splitting-gcf \  
  --source . \  
  --function helloWorld \  
  --region $REGION \  
  --no-allow-unauthenticated \  
  --update-env-vars BG_COLOR=$BG_COLOR
```

如要部署為 Cloud Functions 第 2 代，請使用下列指令：

```
# update the env var
BG_COLOR=tan

gcloud functions deploy traffic-splitting-gcf \
  --gen2 \
  --runtime nodejs20 \
  --entry-point helloHttp \
  --source . \
  --region $REGION \
  --trigger-http \
  --no-allow-unauthenticated \
  --update-env-vars BG_COLOR=$BG_COLOR
```

現在使用 curl 指令連線至端點時，您會看到淺棕色背景。

```
curl -H "Authorization: bearer $(gcloud auth print-identity-token)" -X GET
$SERVICE_URL
```

將流量平均分配

如要在深海綠和棕褐色修訂版本之間分配流量，您需要找出基礎 Cloud Run 服務的修訂版本 ID。執行下列指令即可查看修訂版本 ID：

```
gcloud run revisions list --service traffic-splitting-gcf \
  --region $REGION --format 'value(REVISION)'
```

畫面上會顯示類似下方的結果

```
traffic-splitting-gcf-00003-qoq
traffic-splitting-gcf-00002-zag
```

執行下列指令，即可在兩個修訂版本之間平均分配流量：

```
gcloud run services update-traffic traffic-splitting-gcf \
  --region $REGION \
  --to-revisions <REVISION1>=50,<REVISION2>=50
```

測試流量拆分

您可以前往函式的公開網址 (透過 curl 或直接在瀏覽器中)，測試函式。

```
curl -H "Authorization: bearer $(gcloud auth print-identity-token)" $SERVICE_URL
```

您應該會看到深海綠色修訂版本，另一半則會看到棕褐色修訂版本。輸出內容中也會列出修訂版本名稱，例如：


```
<html><body style="background-color:tan;"><div><p>Hello traffic-splitting-gcf-00006-  
qoq</p></div></body></html>
```

4. 逐步推出

在本節中，您將瞭解如何逐步將變更推出至新的 Cloud Functions 修訂版本。如要進一步瞭解逐步推出功能，請參閱[說明文件](#)。

您會使用與上一節相同的程式碼，但會將其部署為新的 Cloud Function。

首先，將背景顏色設為 `beige`，然後部署名為 `gradual-rollouts-gcf` 的函式。

如要將 Cloud Run 函式直接部署至 Cloud Run，請執行下列指令：

```
# update the env var
BG_COLOR=beige

gcloud beta run deploy gradual-rollouts-gcf \
  --source . \
  --function helloWorld \
  --region $REGION \
  --no-allow-unauthenticated \
  --update-env-vars BG_COLOR=$BG_COLOR
```

如要部署為 Cloud Functions 第 2 代，請使用下列指令：

```
# update the env var
BG_COLOR=beige

# deploy the function
gcloud functions deploy gradual-rollouts-gcf \
  --gen2 \
  --runtime nodejs20 \
  --entry-point helloHttp \
  --source . \
  --region $REGION \
  --trigger-http \
  --no-allow-unauthenticated \
  --update-env-vars BG_COLOR=$BG_COLOR
```

假設我們現在要逐步推出新修訂版本，並將背景顏色設為薰衣草色。

首先，請將目前的修訂版本「米色」設為接收 100% 流量。這樣一來，日後部署的 Cloud 函式就不會收到任何流量。根據預設，Cloud Functions 會將 100% 的流量設定為具有 `latest` 旗標的修訂版本。手動指定目前的米色修訂版本應接收所有流量後，帶有 `latest` 旗標的修訂版本就不會再接收 100% 的流量。詳情請參閱[說明文件](#)。

```
# get the revision name

BEIGE_REVISION=$(gcloud run revisions list --service gradual-rollouts-gcf \
  --region $REGION --format 'value(REVISION)')

# now set 100% traffic to that revision

gcloud run services update-traffic gradual-rollouts-gcf --to-
revisions=$BEIGE_REVISION=100 --region $REGION
```

您會看到類似 `Traffic: 100% gradual-rollouts-gcf2-00001-yox` 的輸出內容

現在您可以部署不會接收任何流量的新修訂版本。您不必變更任何程式碼，只要更新這個修訂版本的 `BG_COLOR` 環境變數即可。

如要將 Cloud Run 函式直接部署至 Cloud Run，請執行下列指令：

```
# update color

BG_COLOR=lavender

# deploy the function that will not receive any traffic
gcloud beta run deploy gradual-rollouts-gcf \
  --source . \
  --function helloWorld \
  --region $REGION \
  --no-allow-unauthenticated \
  --update-env-vars BG_COLOR=$BG_COLOR
```

如要部署為 Cloud Functions 第 2 代，請使用下列指令：

```
# update color

BG_COLOR=lavender

# deploy the function that will not receive any traffic
gcloud functions deploy gradual-rollouts-gcf \
  --gen2 \
  --runtime nodejs20 \
  --entry-point helloHttp \
  --source . \
  --region $REGION \
  --trigger-http \
  --no-allow-unauthenticated \
  --update-env-vars BG_COLOR=$BG_COLOR \
  --tag $BG_COLOR
```

現在請更新 `SERVICE_URL` 環境變數，改用 `gradual-rollouts-gcf` 函式

```
SERVICE_URL=$(gcloud run services describe gradual-rollouts-gcf --platform managed --region $REGION --format 'value(status.url)')
```

現在使用 curl 服務時

```
curl -H "Authorization: bearer $(gcloud auth print-identity-token)" -X GET $SERVICE_URL
```

即使薰衣草色是最近部署的修訂版本，您也會看到米色。

```
<html><body style="background-color:beige;"><div><p>Hello from gradual-rollouts-gcf-24jan16-staging-2-00001-kop using color beige</p></div></body></html>
```

測試提供 0% 流量的修訂版本

假設您已確認修訂版本部署成功，且放送的流量為 0%。即使通過健康狀態檢查，您仍想確認這個修訂版本是否使用薰衣草色背景。

若要測試薰衣草修訂版本，可以[將標記套用至該修訂版本](#)。標記可讓您在特定網址直接測試新修訂版本，而不提供流量。

首先，請取得該修訂版本的圖片網址。

```
IMAGE_URL=$(gcloud run services describe gradual-rollouts-gcf --region $REGION --format 'value(IMAGE)')
```

現在為該圖片加上相關聯的顏色標記。

```
gcloud run deploy gradual-rollouts-gcf --image $IMAGE_URL --no-traffic --tag $BG_COLOR --region $REGION --no-allow-unauthenticated
```

您會看到類似下方的輸出內容：

```
The revision can be reached directly at https://lavender---gradual-rollouts-gcf-k6msmyp47q-lz.a.run.app
```

現在您可以直接捲動這個修訂版本

```
curl -H "Authorization: bearer $(gcloud auth print-identity-token)" -X GET <DIRECT_REVISION_URL>
```

並在結果中看到薰衣草色：

```
<html><body style="background-color:lavender;"><div><p>Hello from gradual-rollouts-gcf-24jan16-00003-xik using color lavender</p></div></body></html>
```

逐步增加流量

現在，您可以開始將流量傳送至薰衣草修訂版本。以下範例說明如何將 1% 的流量傳送至薰衣草。

```
gcloud run services update-traffic gradual-rollouts-gcf --region $REGION --to-tags  
lavender=1
```

如要將 50% 的流量傳送至薰衣草色，可以使用相同指令，但指定 50%。

```
gcloud run services update-traffic gradual-rollouts-gcf --region $REGION --to-tags  
lavender=50
```

畫面上應會顯示各修訂版本接收的流量。

```
Traffic:  
50% gradual-rollouts-gcf-00001-hos  
50% gradual-rollouts-gcf-00004-mum  
    lavender: https://lavender---gradual-rollouts-gcf-k6msmyp47q-uc.a.run.app
```

準備好全面推出薰衣草色時，您可以將薰衣草色設為 100%，取代米色。

```
gcloud run services update-traffic gradual-rollouts-gcf --region $REGION --to-tags  
lavender=100
```

現在造訪或捲動 gradual-rollouts-gcf 函式服務網址時，

```
curl -H "Authorization: bearer $(gcloud auth print-identity-token)" -X GET  
$SERVICE_URL
```

只會看到薰衣草色。

```
<html><body style="background-color:lavender;"><div><p>Hello gradual-rollouts-gcf-  
00004-mum</p></div></body></html>
```

步驟 5 · 約 0 分鐘

5. 復原

假設您收到早期使用者體驗意見回饋，指出顧客偏好米色而非薰衣草色，因此您需要還原為米色。

執行這項指令即可復原到先前的修訂版本 (米色)。

```
gcloud run services update-traffic gradual-rollouts-gcf --region $REGION --to-revisions $BEIGE_REVISION=100
```

現在，當您使用 curl 或造訪函式網址端點時，

```
curl -H "Authorization: bearer $(gcloud auth print-identity-token)" -X GET $SERVICE_URL
```

你會看到米色。

```
<html><body style="background-color:beige;"><div><p>Hello gradual-rollouts-gcf-00001-hos</p></div></body></html>
```

詳情請參閱[說明文件](#)。

步驟 6 · 約 0 分鐘

6. 恭喜！

恭喜您完成本程式碼研究室！

建議您參閱[推出、回溯及流量遷移](#)相關說明文件

涵蓋內容

- 如何在 Cloud Run 函式的兩個或多個修訂版本之間拆分流量
 - 如何逐步推出新修訂版本
 - 如何復原為先前的修訂版本
-

7. 清除所用資源

為避免產生非預期費用 (例如，如果這個 Cloud Run 函式遭非預期呼叫的次數，超過[免費層級每月 Cloud Functions 呼叫次數配額](#))，您可以刪除 Cloud Run 函式，或刪除步驟 2 中建立的專案。

如要刪除部署至 Cloud Run 的 Cloud Run 函式，請前往 Cloud Console 的 Cloud Run 頁面 (<https://console.cloud.google.com/functions/>)，然後刪除您在本程式碼研究室中建立的函式。

如要刪除以第 2 代函式形式部署的 Cloud Run functions，請前往 Cloud Console 的 Cloud Functions 頁面 (<https://console.cloud.google.com/functions/>)，然後刪除您在本程式碼研究室中建立的函式。

如要刪除整個專案，請前往 <https://console.cloud.google.com/cloud-resource-manager>，選取您在步驟 2 中建立的專案，然後選擇「刪除」。刪除專案後，您必須在 Cloud SDK 中變更專案。如要查看所有可用專案的清單，請執行 `gcloud projects list`。
